



Ingress

Огляд мереж Kubernetes

- Мережа Kubernetes дозволяє взаємодіяти між контейнерами, сервісами та зовнішніми користувачами. Це важливо для розподілених застосунків, оскільки забезпечує безперебійний потік даних всередині кластера та за його межами. На відміну від традиційних систем, Kubernetes абстрагує мережу на різні рівні, щоб забезпечити зв'язок між вузлами, подами та зовнішніми середовищами. Основні концепції включають поди, сервіси та мережеві політики. Kubernetes забезпечує плоску мережу, тобто всі поди можуть взаємодіяти один з одним без використання перетворення мережевих адрес (NAT). Такий підхід спрощує налаштування та зменшує накладні витрати на управління внутрішнім зв'язком між сервісами.

Проблеми мережевої взаємодії в розподіленому середовищі

У розподіленій системі, такій як Kubernetes, керування мережею може бути складним. Контейнери, які є тимчасовими, вимагають гнучкої мережі, яка адаптується під час їх створення, видалення та переміщення. Проблеми включають забезпечення узгодженості мережі, обробку перезапусків контейнерів та ефективну маршрутизацію трафіку. Крім того, багатокористувацькі середовища створюють проблеми безпеки, оскільки різним користувачам можуть знадобитися ізольовані мережі. Kubernetes вирішує ці проблеми, абстрагуючи мережу в сервіси та політики, що спрощує маршрутизацію та забезпечує масштабованість. Однак, складність виникає в управлінні цими абстракціями, особливо під час інтеграції зовнішніх сервісів або забезпечення безпечного зв'язку між кластерами.

Мережеві компоненти в Kubernetes

Kubernetes використовує кілька ключових компонентів для керування мережею в кластері. Найфундаментальнішим блоком є Pod, який містить один або кілька контейнерів, що використовують той самий мережевий простір імен. Сервіси використовуються для розкриття Pod-ів, що робить їх видимими як всередині, так і зовні. Типи сервісів включають ClusterIP (для внутрішнього доступу), NodePort та LoadBalancer (для зовнішнього доступу). Мережеві політики керують тим, як Pod-и можуть взаємодіяти один з одним, забезпечуючи безпечний потік даних між компонентами. Разом ці компоненти утворюють гнучку та масштабовану мережу, що дозволяє мікросервісам функціонувати в високорозподіленому середовищі.

Чому традиційних балансувальників навантаження недостатньо

Традиційні балансувальники навантаження розподіляють трафік між екземплярами на основі їхніх IP-адрес або DNS-імен. Хоча цей підхід добре працює для монолітних застосунків, він не підходить для контейнеризованих, динамічних середовищ, таких як Kubernetes, де поди часто змінюють IP-адреси та швидко масштабуються. Традиційні балансувальники навантаження не призначені для обробки постійної зміни екземплярів контейнерів. Kubernetes вирішує цю проблему за допомогою Ingress, який забезпечує більш просунуту маршрутизацію, SSL-термінацію та маршрутизацію на основі шляхів. Ingress може обробляти динамічний характер контейнеризованих застосунків та пропонує більшу гнучкість в управлінні потоком трафіку між мікросервісами.

Потреба в проникненні

Хоча сервіси Kubernetes надають Pods внутрішньому та зовнішньому трафіку, вони не надають розширених функцій маршрутизації трафіку, таких як SSL-термінація, балансування навантаження або маршрутизація на основі шляхів. Саме тут і з'являється Ingress. Ingress — це ресурс Kubernetes, який керує зовнішнім доступом до сервісів, пропонуючи розширений контроль над тим, як трафік надходить до кластера. Він дозволяє маршрутизацію на основі URL-шляхів, хостів або обох, що дозволяє вам надавати доступ кільком сервісам, використовуючи одну IP-адресу. Ingress спрощує складні сценарії управління трафіком, полегшуючи масштабування та захист ваших застосунків Kubernetes. Це важливий інструмент для керування зовнішнім доступом у сучасних мікросервісних архітектурах.

Визначення входу

- Ingress у Kubernetes – це об'єкт API, який керує зовнішнім доступом до сервісів у кластері, зазвичай HTTP або HTTPS. На відміну від Сервісів, які безпосередньо надають Pods зовнішньому трафіку, Ingress дозволяє детально контролювати, як зовнішні запити маршрутизуються до внутрішніх сервісів. Ingress надає розширені функції керування трафіком, такі як маршрутизація на основі URL-адрес, SSL-термінація та балансування навантаження. За допомогою Ingress можна надавати доступ кільком сервісам, використовуючи одну IP-адресу або домен, що зменшує складність обробки зовнішнього трафіку. Він легко інтегрується з контролерами Ingress, які відповідають за реалізацію правил, зазначених в об'єктах Ingress.

Роль вхідного доступу в управлінні зовнішнім доступом

- Ingress відіграє вирішальну роль в управлінні взаємодією зовнішніх користувачів із сервісами всередині кластера Kubernetes. Виступаючи шлюзом, Ingress дозволяє розробникам визначати правила для спрямування вхідного HTTP- або HTTPS-трафіку до відповідного сервісу на основі таких факторів, як URL-шлях або ім'я хоста. Це усуває необхідність безпосереднього надання кожному сервісу окремих IP-адрес або балансувальників навантаження, зменшуючи складність інфраструктури та вартість. Крім того, Ingress може обробляти SSL/TLS-термінацію, забезпечуючи безпечний зв'язок між користувачами та програмами. Він забезпечує централізоване керування маршрутизацією трафіку та безпекою, що робить його життєво важливим компонентом мереж Kubernetes.

Різниця між сервісами та вхідним контентом

- Хоча і Сервіси, і Ingress надають зовнішньому трафіку застосункам Kubernetes, вони виконують різні функції. Сервіси, такі як NodePort та LoadBalancer, спрямовують трафік безпосередньо до Pod, але не мають розширених можливостей маршрутизації та управління трафіком. Вони прості у використанні, але можуть стати неефективними під час керування багатьма сервісами. Ingress, з іншого боку, дозволяє використовувати складні правила маршрутизації, такі як спрямування трафіку на основі URL-шляхів або імен хостів. Ingress також підтримує SSL-термінацію та маршрутизацію на основі шляхів, що робить його більш гнучким та масштабованим для складних застосунків на основі мікросервісів. Це зменшує потребу в кількох сервісах LoadBalancer, заощаджуючи на хмарних витратах.

Ключові характеристики Ingress

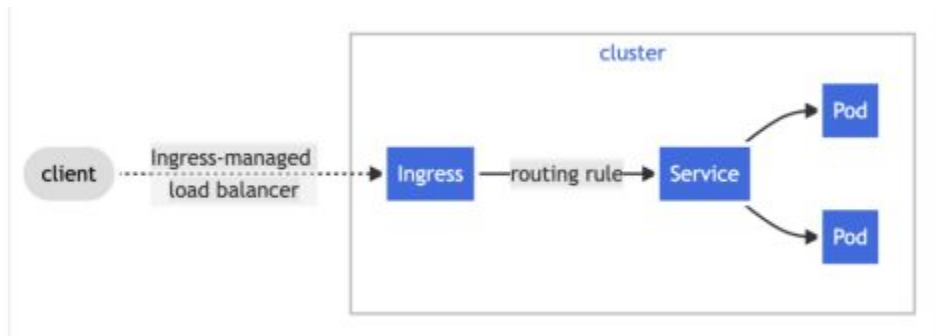
- Ingress надає кілька ключових функцій, які роблять його потужним інструментом для керування зовнішнім доступом до сервісів Kubernetes. Маршрутизація на основі шляхів дозволяє надавати доступ різним сервісам на основі URL-шляхів (наприклад, /app1, /app2). Маршрутизація на основі хоста дозволяє користувачам маршрутизувати трафік на основі доменних імен (наприклад, app1.example.com, app2.example.com). Ingress також підтримує SSL/TLS-термінацію, що забезпечує безпечний зв'язок між клієнтами та сервісами. Крім того, він дозволяє балансувати навантаження, розподіляючи трафік між кількома серверними сервісами для кращої продуктивності та резервування. Ці функції роблять Ingress незамінним для сучасних розподілених програм, які потребують динамічного та безпечного управління трафіком.

Приклад простого вхідного ресурсу

```
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: example-ingress
5  spec:
6    rules:
7      - host: "example.com"
8        http:
9          paths:
10             - path: /app1
11               pathType: Prefix
12               backend:
13                 service:
14                   name: app1-service
15                   port:
16                     number: 80
17             - path: /app2
18               pathType: Prefix
19               backend:
20                 service:
21                   name: app2-service
22                   port:
23                     number: 80
24
```

Що таке контролер вхідного доступу?

Контролер Ingress – це компонент Kubernetes, відповідальний за реалізацію правил, визначених у ресурсах Ingress. Він відстежує зміни в об'єктах Ingress та оновлює свою внутрішню конфігурацію маршрутизації, щоб відповідно спрямувати зовнішній трафік. Контролери Ingress зазвичай розгортаються як поди (Pods) у кластері Kubernetes та обробляють маршрутизацію, завершення SSL, балансування навантаження та інші завдання управління трафіком. Вони діють як місток між зовнішнім трафіком та сервісами Kubernetes. Без контролера Ingress об'єкти Ingress не можуть функціонувати, оскільки контролер забезпечує дотримання правил та керує фактичним потоком трафіку в кластері.



Популярні контролери вхідного доступу



Існує кілька популярних контролерів вхідного сигналу, кожен з яких підходить для різних середовищ та випадків використання. Деякі з найбільш широко використовуваних:

- **Контролер вхідного трафіку NGINX:** Широко поширений контролер, простий у використанні, з акцентом на балансування навантаження та управління трафіком.
- **Traefik:** Відомий своєю простотою налаштування, динамічним виявленням сервісів та вбудованою підтримкою SSL-сертифікатів Let's Encrypt.
- **HAProxy:** Високопродуктивний контролер з надійними функціями маршрутизації, SSL-термінації та балансування навантаження.
- **Kong:** Потужний шлюз API, який пропонує розширені функції, такі як автентифікація, обмеження швидкості та управління API, поряд зі стандартними функціями Ingress. Кожен контролер має свої унікальні переваги, залежно від складності та потреб ваших застосунків.

Відмінності між контролерами вхідного доступу

Хоча всі контролери Ingress виконують схожі функції, між ними є помітні відмінності:

- **Продуктивність:** Контролери, такі як HAProxy, чудово працюють у високопродуктивних середовищах, тоді як Traefik та NGINX підходять для загальних випадків використання.
- **Простота використання:** Traefik простіший у налаштуванні завдяки динамічному виявленню сервісів, тоді як NGINX та HAProxy можуть вимагати більше ручного налаштування.
- **Функції:** Kong пропонує додаткові функції керування API, такі як автентифікація та обмеження швидкості, які недоступні у стандартних контролерах.
- **Підтримка SSL:** Traefik вирізняється автоматичним керуванням SSL-сертифікатами за допомогою Let's Encrypt, тоді як інші контролери можуть вимагати ручного налаштування. Ці відмінності впливають на те, який контролер найкраще підходить для конкретних робочих навантажень Kubernetes.

Анатомія YAML-файлу вхідного ресурсу

Ресурс Ingress у Kubernetes визначається за допомогою YAML-файлу, який визначає, як зовнішні запити повинні спрямовуватися до внутрішніх сервісів. Ключові компоненти включають метадані, специфікацію та правила. Розділ метаданих містить таку інформацію, як ім'я та простір імен Ingress. Розділ специфікації визначає правила маршрутизації, які можуть включати маршрутизацію на основі шляху або хоста. Кожне правило складається з хоста, шляхів та серверної служби, яка отримує трафік. За потреби можна додавати анотації для налаштування певної поведінки, такої як правила завершення SSL або перезапису. Ця структура файлу забезпечує дуже гнучкий спосіб керування зовнішнім доступом до кількох сервісів.

Базова конфігурація вхідного трафіку (маршрутизація на основі шляху, маршрутизація на основі хоста)

Ingress дозволяє використовувати два основні типи маршрутизації: на основі шляху та на основі хоста. При маршрутизації на основі шляху трафік маршрутизується на основі URL-шляху. Наприклад, `/app1` може маршрутизувати до `app1-service`, тоді як `/app2` маршрутизує до `app2-service`. При маршрутизації на основі хоста трафік маршрутизується на основі імені хоста або домену, наприклад, `app1.example.com` та `app2.example.com`. Це спрощує керування кількома сервісами під однією IP-адресою. Базові конфігурації зазвичай включають визначення правил в Ingress YAML, які визначають, які шляхи або хости маршрутизуються до яких серверних сервісів.

Маршрутизація на основі імені хоста

Ingress підтримує маршрутизацію на основі хоста, де запити спрямовуються до різних сервісів на основі запитуваного доменного імені. Наприклад, трафік для `app1.example.com` може бути спрямований до `app1-service`, тоді як `app2.example.com` спрямовується до `app2-service`. Це налаштовується шляхом визначення правил у ресурсі Ingress за допомогою поля `host`:

```
1 rules:
2   - host: "app1.example.com"
3     http:
4       paths:
5         - path: /
6           backend:
7             service:
8               name: app1-service
9               port:
10                number: 80
11  - host: "app2.example.com"
12    http:
13      paths:
14        - path: /
15          backend:
16            service:
17              name: app2-service
18              port:
19                number: 80
20
```

Маршрутизація на основі URL-шляху

Маршрутизація на основі шляху дозволяє спрямовувати трафік до різних сервісів на основі URL-шляху запити. Це особливо корисно для мікросервісів, де різні програми можуть використовувати один і той самий домен, але їм потрібно обробляти трафік на різних шляхах. Ось приклад конфігурації маршрутизації на основі шляху:

```
1  rules:
2    - host: "example.com"
3      http:
4        paths:
5          - path: /app1
6            pathType: Prefix
7            backend:
8              service:
9                name: app1-service
10               port:
11                 number: 80
12          - path: /app2
13            pathType: Prefix
14            backend:
15              service:
16                name: app2-service
17                port:
18                  number: 80
```

Використання регулярних виразів для розширеної маршрутизації

- Для розширених сценаріїв маршрутизації Ingress може використовувати регулярні вирази (regex) для зіставлення складних шаблонів URL-адрес. Це забезпечує більшу гнучкість маршрутизації, дозволяючи вам зіставляти кілька шаблонів URL-адрес за допомогою одного правила. Наприклад, якщо ви хочете спрямувати трафік для будь-якого шляху, який починається з /api, до певного сервісу, ви можете використовувати шаблон регулярного виразу:

```
1  nginx.ingress.kubernetes.io/use-regex: "true"
2  rules:
3    - host: "example.com"
4      http:
5        paths:
6          - path: /api/*
7            pathType: ImplementationSpecific
8            backend:
9              service:
10               name: api-service
11               port:
12                 number: 80
```

Приклад сценарію: маршрутизація на основі хоста

У типовому сценарії маршрутизації на основі хоста, один ресурс Ingress може керувати трафіком для кількох доменів. Наприклад, якщо у вас є дві програми, що працюють на одному кластері Kubernetes, одна для `blog.example.com`, а інша для `shop.example.com`, ви можете налаштувати ресурс Ingress наступним чином:

```
1  rules:
2    - host: "blog.example.com"
3      http:
4        paths:
5          - path: /
6            backend:
7              service:
8                name: blog-service
9                port:
10               number: 80
11   - host: "shop.example.com"
12     http:
13       paths:
14         - path: /
15           backend:
16             service:
17               name: shop-service
18               port:
19                 number: 80
20
```

Приклад сценарію: Маршрутизація на основі шляху

У сценарії маршрутизації на основі шляху трафік спрямовується до різних сервісів на основі URL-шляху. Розглянемо ситуацію, коли у вас є два сервіси, один для керування користувачами, а інший для платежів, обидва працюють в одному домені. Ви можете налаштувати ресурс Ingress таким чином:

```
1 rules:
2   - host: "example.com"
3     http:
4       paths:
5         - path: /users
6           pathType: Prefix
7           backend:
8             service:
9               name: user-service
10              port:
11                number: 80
12         - path: /payments
13           pathType: Prefix
14           backend:
15             service:
16               name: payment-service
17              port:
18                number: 80
19
```

Питання та відповіді